مفهوم العملية(Process Concept)

🎯 نظرة سريعة: شنو راح تتعلم؟

هذا الموضوع يسولف عن "العملية" اللي هي "عقل" الحاسبة الشغال. العملية هي البرنامج بس لما يصير نشط ويشتغل بالذاكرة، مو مجرد ملف خامل بالقرص.

الأهداف:

- ✅ تفهم الفرق بين البرنامج (Passive) والعملية (Active)
- ✅ تعرف أجزاء العملية بالذاكرة (Text, Data, Heap, Stack)
- ✅ تحفظ حالات العملية الخمسة وتحولاتها
- ✅ تفهم محتويات PCB ودوره في إدارة العمليات

مدة المراجعة المقترحة: 45 دقيقة 📝

📖 القسم الأول: المفاهيم الأساسية (حفظ + فهم)

🔑 التعريفات المهمة

البرنامج (Program):

- **التعريف:** كيان خامل (Passive entity) يحتوي على قائمة تعليمات مخزونة كملف تنفيذي
- **ببساطة:** مثل "قرص سي دي" عليه لعبة، جالس بالرف ما يسوي شي
- **مثال:** ملف Word.exe الموجود بالقرص الصلب
- ⚠️ **انتبه:** البرنامج الواحد ما يستهلك موارد لحد ما يشتغل

العملية (Process):

- **التعريف:** برنامج في حالة تنفيذ (A program in execution) - كيان نشط (Active entity)
- **ببساطة:** لما تضغط مرتين بالماوس على اللعبة وتشتغل، صارت "عملية"
- **الفرق عن البرنامج:** العملية إلها Program Counter و PCB و موارد مخصصة
- ⭐ **نقطة ذهبية:** برنامج واحد يقدر ينتج عدة عمليات (مثل فتح عدة نوافذ Chrome)

كتلة التحكم بالعملية (PCB - Process Control Block):

- **التعريف:** مخزن معلومات يحتوي كل بيانات العملية (يسمى أيضاً Task Control Block)
- **ببساطة:** مثل "دفتر الملاحظات" اللي يخلي نظام التشغيل يتذكر وين وصلت كل عملية
- **مثال:** لما النظام يوكف عمليتك ويشغل عملية ثانية، يحفظ كل شي بالـ PCB حتى يرجع يكمل
- ⚠️ **انتبه:** PCB موجود بالذاكرة الرئيسية (Main Memory) مو بالقرص

عداد البرنامج (Program Counter):

- **التعريف:** سجل يحتوي عنوان الأمر التالي اللي راح ينفذ
- ⭐ **نقطة ذهبية:** هذا أهم جزء بالـ PCB - يگول للنظام "وين وصلنا بأي سطر بالكود؟"

الخيوط (Threads):

- **التعريف:** خيوط تنفيذ متعددة داخل نفس العملية
- **ببساطة:** ببرنامج Word، خيط يخليك تطبع، وخيط ثاني يدقق إملاء بنفس الوقت
- **الفرق عن Process:** الخيوط تتشارك نفس الذاكرة، العمليات كل واحدة إلها ذاكرتها

📊 جدول الفروقات (مهم جداً للـ MCQ!)

الميزة	البرنامج (Program)	العملية (Process)
الطبيعة	كيان خامل (Passive)	كيان نشط (Active)
الموقع	القرص الصلب (Secondary Memory)	الذاكرة الرئيسية (Main Memory)
الموارد	ما يستهلك موارد	مخصص لها CPU، ذاكرة، I/O
عداد البرنامج	❌ ما عنده	✅ عنده Program Counter
الحالة	ثابت	متغيرة (New, Ready, Running, إلخ)
التعدد	ملف واحد	يقدر ينتج عدة عمليات من نفس البرنامج

⭐ **للحفظ السريع:**

- البرنامج = **ملف خامل بالقرص**
- العملية = **برنامج شغال بالذاكرة**

📊 أجزاء العملية بالذاكرة

القسم	المحتوى	الحجم	الاستخدام
Text	كود البرنامج القابل للتنفيذ	ثابت	الأوامر اللي راح تنفذ
Data	المتغيرات العالمية (Global Variables)	ثابت	البيانات المشتركة
Heap	ذاكرة ديناميكية (malloc, new)	ديناميكي ↑	التخصيص أثناء التشغيل
Stack	معاملات الدوال، متغيرات محلية، عناوين الرجوع	ديناميكي ↓	استدعاء الدوال

⭐ للحفظ:

- Text + Data = **ثابتين**
- Stack + Heap = **ديناميكيين** (ينمون بعكس بعض)
- ⚠️ **فخ:** لو التقى Stack مع Heap = كارثة (Stack Overflow)!

⚙️ حالات العملية (Process States) - حفظ واجب!

الحالات الخمسة:

1. **New (جديدة):** العملية لسه دتتحمل للنظام
2. **Ready (جاهزة):** واكفة بالسرة، تنتظر المعالج يفرغ
3. **Running (مشغولة):** هسة هي جوه يد المعالج ويدفذ أوامرها
4. **Waiting/Blocked (انتظار):** واكفة تنتظر شي يخلص (I/O، حدث معين)
5. **Terminated (منتهية):** خلصت شغلها وطلعت من النظام

التحولات بين الحالات:

```text
New → Ready: النظام خصص لها موارد
Ready → Running: الـ Scheduler اختارها
Running → Ready: انتهى وقتها (Time Quantum) أو جاءت عملية أعلى أولوية [web:11]
Running → Waiting: طلبت I/O أو تنتظر حدث
Waiting → Ready: انتهى I/O أو الحدث صار
Running → Terminated: خلصت تنفيذ أو صار Error
```

⭐ **قاعدة ذهبية:** بأي لحظة، عملية **واحدة بس** تكون Running على كل Core، بس ممكن ألاف بحالة Ready أو Waiting

📦 محتويات PCB (أهم جزء للامتحان!)

PCB يحتوي على:

1. **Process ID (PID):** رقم تعريف العملية (مثل رقم الهوية)
2. **Process State:** الحالة الحالية (New, Ready, Running، إلخ)
3. **Program Counter:** عنوان السطر الجاي للتنفيذ
4. **CPU Registers:** Accumulators، Stack Pointers، Index Registers
5. **CPU Scheduling Info:** الأولوية، مؤشرات طوابير الجدولة
6. **Memory Management Info:** Base register، Limit register، Page tables
7. **I/O Status Info:** قائمة الملفات المفتوحة والأجهزة المخصصة
8. **Accounting Info:** وقت التنفيذ، الحدود الزمنية

⭐ **أهمية PCB:** يخلي النظام يسوي Context Switch - يوكف عملية ويحفظ حالتها، ثم يرجع لها بالضبط من وين واكف

💡 القسم الثاني: السيناريوهات (أسئلة "ماذا لو")

🤔 السيناريو 1: فتح نوافذ متعددة من نفس البرنامج

الموقف:

فتحت Google Chrome 3 مرات (3 نوافذ منفصلة)

السؤال: هل هذه عملية واحدة أم ثلاث عمليات؟

الجواب:

- برنامج واحد: ✅ صح (Chrome.exe ملف واحد)
- عملية واحدة: ❌ خطأ
- **ثلاث عمليات مستقلة:** ✅ **صح** - كل نافذة لها PID و PCB خاص
- تتشارك الذاكرة: ❌ خطأ - كل عملية لها مساحة ذاكرة منفصلة

⭐ **القاعدة:** برنامج واحد يقدر ينتج عمليات متعددة، كل واحدة مستقلة

🤔 السيناريو 2: انتقال العملية من Running إلى Waiting

الموقف:

عملية P1 تشتغل على CPU وطلبت قراءة ملف من القرص الصلب

السؤال: شنو يصير للعملية؟

الجواب:

- تبقى Running: ❌ خطأ - CPU ما ينتظر I/O (بطيء جداً)
- تروح Terminated: ❌ خطأ - مو منتهية، بس تنتظر
- **تروح Waiting:** ✅ **صح** - تنتظر انتهاء I/O
- CPU يبقى فاضي: ❌ خطأ - Scheduler يشغل عملية ثانية من Ready

⭐ **القاعدة:** أي طلب I/O يحول العملية من Running → Waiting، والـ CPU يشتغل على عملية ثانية

🤔 السيناريو 3: تخصيص ذاكرة ديناميكية

الموقف:

برنامج يستخدم `()malloc` لتخصيص 1000 بايت أثناء التشغيل

السؤال: من أي قسم بالذاكرة تأتي هذه الذاكرة؟

الجواب:

- Text section: ❌ خطأ - هذا للكود
- Data section: ❌ خطأ - للمتغيرات العالمية الثابتة
- Stack: ❌ خطأ - للمتغيرات المحلية والدوال
- **Heap**: ✅ **صح** - التخصيص الديناميكي يأتي من الـ Heap

⭐ **القاعدة:** Stack | global variables → Data → متغيرات محلية | malloc/new → Heap

---

🤔 السيناريو 4: **Context Switch**

الموقف:

نظام التشغيل قرر ينقل CPU من عملية P1 لعملية P2

السؤال: شنو يصير بالضبط؟

الجواب:

الخطوات:

1. حفظ حالة P1 (Program Counter، Registers) في PCB مالتها
2. تحديث حالة P1 من Running → Ready (أو Waiting)
3. تحميل حالة P2 من PCB مالتها
4. تحديث حالة P2 من Ready → Running
5. المعالج يكمل تنفيذ P2 من وين واكف

⚠️ **انتبه:** Context Switch هو "وقت ضائع" (Overhead) - النظام ما يسوي شغل مفيد، بس يبدل العمليات

⭐ **القاعدة:** PCB هو المفتاح اللي يخلي Context Switch يصير بدون ما تضيع المعلومات

🤔 السيناريو 5: عملية في حالة Waiting

الموقف:

عملية P1 في حالة Waiting تنتظر انتهاء قراءة ملف، والملف خلص

السؤال: وين تروح العملية؟

الجواب:

- Waiting → Running مباشرة: ❌ **فخ!** - هذا الخطأ الشائع
- **Waiting → Ready**: ✅ **صح** - تدخل طابور الانتظار
- بعدها Scheduler يختارها: ✅ من Ready → Running

⭐ **القاعدة: ما في انتقال مباشر** من Waiting إلى Running - لازم تمر بـ Ready

---

التعريفات في 30 ثانية

```text
Program = ملف خامل بالقرص
Process = برنامج نشط بالذاكرة
PCB = دفتر ملاحظات العملية
Program Counter = عنوان الأمر التالي
Context Switch = تبديل العمليات (Overhead)
Thread = خيط تنفيذ داخل العملية
Stack = متغيرات محلية ودوال
Heap = تخصيص ديناميكي (malloc)
Text = كود البرنامج
Data = متغيرات عالمية
```

القواعد الذهبية (احفظها!)

1. ⭐ **القاعدة 1:** عملية واحدة فقط تكون Running على كل Core
2. ⭐ **القاعدة 2:** Waiting → Ready → Running (ما في اختصار!)
3. ⭐ **القاعدة 3:** Local → Stack | Global → Data | Dynamic → Heap
4. ⭐ **القاعدة 4:** Program Counter = الأمر التالي (Next) مو الحالي
5. ⭐ **القاعدة 5:** برنامج واحد → عمليات متعددة (مو العكس)
6. ⭐ **القاعدة 6:** Stack ↓ و Heap ↑ (بعكس بعض)
7. ⭐ **القاعدة 7:** Text + Data = ثابت | Stack + Heap = ديناميكي
8. ⭐ **القاعدة 8:** PCB بالـ RAM، مو بالقرص

الفروقات السريعة

مقابل	الفرق الرئيسي
Program vs Process	خامل vs نشط
Stack vs Heap	محلي/دوال vs ديناميكي
Text vs Data	كود vs متغيرات عالمية
Ready vs Waiting	تنتظر CPU vs تنتظر I/O
New vs Terminated	بداية vs نهاية
Process vs Thread	ذاكرة منفصلة vs ذاكرة مشتركة

الأخطاء الشائعة - تجنبها!

❌ **خطأ 1:** Waiting → Running مباشرة → ✅ **الصحيح:** Waiting → Ready → Running

❌ **خطأ 2:** PC = الأمر الحالي → ✅ **الصحيح:** PC = الأمر التالي

❌ **خطأ 3:** عدة عمليات Running على Core واحد → ✅ **الصحيح:** عملية واحدة فقط

❌ **خطأ 4:** Global في Global **الصحيح:** ✅ → Stack في Data

❌ **خطأ 5:** PCB بالقرص → ✅ **الصحيح:** PCB بالـ RAM

❌ **خطأ 6:** البرنامج = العملية → ✅ **الصحيح:** البرنامج خامل، العملية نشطة

متى نستخدم إيش؟

```text
محتاج تخزن متغير محلي → Stack
محتاج تخصيص ديناميكي → Heap (malloc)
محتاج متغير عام → Data section
محتاج تخزن كود → Text section
محتاج تتبع العملية → PCB
محتاج تعرف السطر الجاي → Program Counter
محتاج تبدل عمليات → Context Switch
```

جدولة العمليات والعمليات على العمليات

🎯 نظرة سريعة: شنو راح تتعلم؟

بعد ما عرفنا شنو هي العملية، هسة نتعلم شلون نظام التشغيل **يدير** هاي العمليات (Scheduling) وشلون تتكاثر وتموت (Operations).

الأهداف:

- ✅ تفهم الفرق بين Job Queue, Ready Queue, Device Queue
- ✅ تعرف أنواع المجدلات (Long/Short/Medium-term Scheduler)
- ✅ تفهم Context Switch وليش يعتبر Overhead
- ✅ تفرق بين fork() و exec() و CreateProcess()
- ✅ تعرف Parent-Child relationship وشجرة العمليات
- ✅ تميز بين Zombie و Orphan processes

مدة المراجعة المقترحة: 60 دقيقة 📝

📖 القسم الأول: المفاهيم الأساسية

🔑 التعريفات المهمة

طوابير الجدولة (Scheduling Queues):

- **التعريف:** قوائم انتظار لتنظيم العمليات حسب حالتها
- **ببساطة:** مثل "طوابير البنك" - كل واحد ينتظر دوره حسب نوع الخدمة
- ⭐ **نقطة ذهبية:** الطوابير غالباً implemented as **Linked Lists**

Job Queue (طابور الوظائف):

- **التعريف:** يحتوي **كل العمليات** في النظام (حتى اللي على القرص)
- **ببساطة:** هذا الطابور "الأم" - كل عملية جديدة تدخل هنا أول شي
- **الموقع:** على القرص الصلب (Secondary Storage)
- ⚠️ **انتبه:** Job Queue يشمل كل شي - حتى اللي ما دخلت الذاكرة بعد

Ready Queue (طابور الجاهزية):

- **التعريف:** عمليات في الذاكرة الرئيسية، جاهزة للتنفيذ، تنتظر فقط CPU
- **ببساطة:** الناس الواقفين قدام الكاشير مباشرة - مستعدين للدفع
- **الموقع:** في الذاكرة الرئيسية (Main Memory)
- ⭐ **نقطة ذهبية:** إذا كل العمليات I/O bound، Ready Queue تكون **شبه فاضية**

Device Queue (طابور الأجهزة):

- **التعريف:** عمليات تنتظر جهاز I/O معين (قرص صلب، طابعة، إلخ)
- **ببساطة:** واحد راح يصور نسخ بالطابعة وجالس ينتظر
- **مثال:** Disk Queue (انتظار قراءة/كتابة)، Printer Queue

📊 جدول الفروقات الأساسية (حفظ واجب!)

الميزة	Job Queue	Ready Queue	Device Queue
المحتوى	كل العمليات في النظام	عمليات في RAM جاهزة للتنفيذ	عمليات تنتظر I/O
الموقع	القرص + الذاكرة	الذاكرة الرئيسية فقط	الذاكرة الرئيسية
الحالة	كل الحالات	**Ready state** فقط	**Waiting state**
تنتظر إيش؟	دخول النظام	تخصيص CPU	انتهاء I/O
من يديرها؟	Long-term Scheduler	Short-term Scheduler	Device Driver

⭐ **للحفظ السريع:**

- Job Queue = **الأم الكبيرة** (كل شي)
- Ready Queue = **جاهز للشغل** (ينتظر CPU)
- Device Queue = **ينتظر جهاز** (I/O)

⚙️ أنواع المجدلات (Schedulers) - أهم جزء!

1. **Long-term Scheduler (Job Scheduler):**

- **الوظيفة:** يختار عمليات من **Job Queue** → **Ready Queue**
- **ببساطة:** هو "البواب" - يقرر منو يدخل للنظام ومنو لا
- **السرعة: بطيء** (يشتغل كل دقائق أو ثواني)
- **الهدف:** يتحكم بـ **Degree of Multiprogramming** (عدد العمليات بالذاكرة)
- ⭐ **نقطة ذهبية:** ما موجود بالأنظمة الحديثة (مثل Windows, UNIX) - يدخلون كل العمليات مباشرة

2. **Short-term Scheduler (CPU Scheduler):**

- **الوظيفة:** يختار عملية من **CPU** → **Ready Queue**
- **ببساطة:** هو "المايسترو" - يقرر منو يشتغل على CPU هسة
- **السرعة: سريع جداً** (كل 100 milliseconds أو أقل)
- **الهدف:** تعظيم استخدام CPU
- ⚠️ **انتبه:** هذا أهم مجدل - يشتغل باستمرار

3. **Medium-term Scheduler:**

- **الوظيفة: Swapping** - يطلع عمليات من الذاكرة للقرص وبالعكس
- **ببساطة:** "المدير المالي" - يوزن بين العمليات حسب الأولوية
- **السرعة:** متوسطة (بين Long و Short)
- **الهدف:** تقليل **Degree of Multiprogramming** لو الذاكرة امتلأت

🔄 تبديل السياق (Context Switch) - حفظ واجب!

التعريف:

عملية حفظ حالة العملية الحالية في PCB وتحميل حالة عملية جديدة

الخطوات:

1. حفظ **Program Counter** و **Registers** للعملية القديمة في PCB
2. تحديث حالة العملية القديمة (Running → Ready أو Waiting)
3. اختيار عملية جديدة من Ready Queue
4. تحميل **PCB** للعملية الجديدة (PC, Registers، إلخ)
5. تحديث حالة العملية الجديدة (Ready → Running)

⚠️ **أهم نقطة للامتحان:**

Context Switch يعتبر **Overhead (وقت ضائع)** لأن:

- المعالج **ما يسوي شغل حقيقي** للمستخدم
- بس يحفظ ويحمل بيانات إدارية
- يستهلك مئات الـ cycles
- يسبب **TLB miss** و **Cache miss**

⭐ **قاعدة ذهبية:** كل ما زاد عدد Context Switches، قلّ الأداء!

🧒 إنشاء العمليات (Process Creation)

العلاقة الأساسية: Parent-Child

- العملية اللي تنشئ عملية جديدة = **Parent (الأب)**
- العملية الجديدة = **Child (الابن)**
- هذا يكوّن **Process Tree (شجرة عمليات)**

في نظام UNIX/Linux:

fork() System Call:

- **الوظيفة:** ينسخ العملية الحالية - ينتج **نسخة طبق الأصل**
- **Return Value:**
  - في الأب: يرجع **PID الابن** (رقم موجب)
  - في الابن: يرجع **0**
  - في حالة فشل: يرجع **-1**
- **ببساطة:** مثل "الاستنساخ" - ينتج توأم متطابق
- ⚠️ **انتبه:** بعد fork()، الأب والابن يكملون من **نفس السطر**

exec() System Call:

- **الوظيفة:** يستبدل **كود العملية الحالية** ببرنامج جديد
- **Return Value: ما يرجع** إذا نجح! (لأنه استبدل الكود كله)
- **ببساطة:** الابن "يغير ملابسه" - يصير برنامج مختلف تماماً
- ⭐ **نقطة ذهبية:** exec() يحذف Memory (Stack, Heap, Data) ويبني من جديد

في نظام Windows:

CreateProcess():

- **الوظيفة:** ينشئ عملية جديدة **مباشرة** مع تحديد البرنامج من البداية
- **الفرق عن fork:** ما ينسخ العملية الحالية - يبدأ من الصفر
- **ببساطة:** "التوليد المباشر" - تحدد شنو البرنامج من البداية
- ⚠️ **انتبه:** CreateProcess() أكثر صرامة من fork/exec

📊 جدول المقارنة: fork() vs exec() vs CreateProcess()

الميزة	fork()	exec()	CreateProcess()
الوظيفة	ينسخ العملية الحالية	يستبدل الكود بكود جديد	ينشئ عملية جديدة
النظام	UNIX/Linux	UNIX/Linux	Windows
Return Value	PID (للأب) / 0 (للابن)	**لا يرجع** إذا نجح	Handle للعملية
الذاكرة	ينسخ (مع COW)	يحذف ويبني من جديد	جديدة تماماً
الاستخدام	إنشاء نسخة	تحويل لبرنامج آخر	إنشاء مستقل

⭐ **للحفظ:**

- **Cloning** = fork() **(استنساخ)**
- **Replacement** = exec() **(استبدال)**
- **Direct Creation** = CreateProcess() **(إنشاء مباشر)**

🧟 الحالات الخاصة (مهمة جداً للـ MCQ!)

1. Zombie Process (العملية الزومبي):

- **التعريف:** عملية **خلصت تنفيذها** (terminated) بس لسه **إلها entry في Process Table**
- **ببساطة:** "ميتة بس موجودة كأسم" - مثل شبح!
- **السبب:** الأب **ما عمل wait()** حتى يقرأ Exit Status مال الابن
- **الحل:** الأب يستدعي **wait()** لـ "reap" الزومبي
- ⚠️ **الخطر:** لو تجمعت زومبيات كثيرة، تملأ Process Table!

2. Orphan Process (العملية اليتيمة):

- **التعريف:** عملية **أبوها مات** (terminated) وهي **لسه شغالة**
- **ببساطة:** "يتيم" - الأب راح وتركه لحاله
- **الحل:** نظام التشغيل يخلي **init process** (PID=1) "يتبناها"
- ⚠️ **انتبه:** Orphan **مو مشكلة** - النظام يحلها تلقائياً

📊 جدول المقارنة: Zombie vs Orphan

الميزة	Zombie Process	Orphan Process
الحالة	خلصت تنفيذ (Dead)	لسه شغالة (Alive)
الأب	حي بس ما عمل wait()	**مات**
في Process Table	✅ نعم (هنا المشكلة!)	✅ نعم
الموارد	محررة (بس entry باقي)	تستهلك موارد
الحل	الأب يعمل **wait()**	**init** يتبناها تلقائياً
هل مشكلة؟	✅ **نعم** (تملأ الجدول)	❌ لا (النظام يحلها)

💡 القسم الثاني: السيناريوهات

🤔 السيناريو 1: العملية تطلب I/O

الموقف:

عملية P1 في حالة Running وطلبت قراءة ملف من القرص الصلب

السؤال: شنو يصير بالترتيب؟

الجواب:

1. P1 تنتقل من **Running → Waiting**
2. تدخل **Device Queue** (Disk Queue)
3. Short-term Scheduler يختار عملية من **Ready Queue** ويحطها على CPU
4. لما الملف يخلص قراءة، P1 تنتقل **Waiting → Ready**
5. تدخل **Ready Queue** وتنتظر دورها للـ CPU

⚠️ **الفخ:** P1 **ما تروح Running مباشرة** - لازم تمر بـ Ready!

⭐ **القاعدة:** أي طلب I/O → Waiting state → Device Queue

🤔 السيناريو 2: عملية تسوي fork()

الموقف:

```c
int x = 5;
pid_t pid = fork();
if (pid == 0) {
    x = 10;
    printf("Child: x=%d\n", x);
}
else {
    printf("Parent: x=%d\n", x);
}
```

السؤال: شنو راح يطبع كل واحد؟

الجواب:

- **Child:** x=10 (لأنه غيّر قيمة x)
- **Parent:** x=5 (الأب ما تأثر - نسخ منفصلة!)

⭐ **القاعدة:** fork() ينسخ، بس التعديلات ما تنتقل بين الأب والابن (Copy-on-Write)

🤔 السيناريو 3: fork() بدون exec()

الموقف:

عملية Parent عملت fork() بس **ما عملت exec()**

السؤال: شنو راح يصير؟

الجواب:

- الابن راح يكون **نسخة طبق الأصل** من الأب
- راح يكملون **نفس الكود** من السطر بعد fork()
- بس Return value مختلف:
  - الأب يشوف PID الابن (رقم موجب)
  - الابن يشوف 0

⭐ **القاعدة:** fork() بدون exec() = استنساخ | fork() مع exec() = برنامج جديد

🤔 السيناريو 4: الأب ما عمل wait()

الموقف:

```c
pid_t pid = fork();
if (pid > 0) {
    // الأب ما عمل wait()
}
else {
    exit(0);  // الابن خلص
}
```

السؤال: شنو راح يصير للابن؟

الجواب:

- الابن راح يصير **Zombie**
- راح يبقى entry بالـ Process Table
- لحد ما الأب يعمل **wait()** أو **الأب يموت**

⚠️ **الفخ:** بعض الطلاب يعتقدون الابن يختفي تلقائياً - لا!

⭐ **القاعدة:** Child terminates + Parent doesn't wait() = Zombie

🤔 السيناريو 5: كل العمليات **I/O bound**

الموقف:

في نظام، كل العمليات I/O bound (تقضي وقت طويل بالـ I/O)

السؤال: شنو راح يصير للـ Ready Queue؟

الجواب:

- Ready Queue راح تكون **شبه فاضية** (almost empty)
- لأن العمليات راح تقضي أغلب وقتها في (Waiting) **Device Queues**
- Short-term Scheduler راح يكون عنده **شغل قليل**

⭐ **القاعدة:** I/O bound processes → Ready Queue empty | CPU bound → Ready Queue full

⚠️ القسم الرابع: الأخطاء الشائعة والفخوخ

فخ 1: **Job Queue** تحتوي بس العمليات الجديدة

الخطأ الشائع:

"Job Queue فقط للعمليات الجديدة (New state)"

الصحيح:

Job Queue تحتوي **كل العمليات في النظام** - بكل الحالات

فخ 2: **exec()** يرجع قيمة

الخطأ الشائع:

"exec() يرجع PID أو 0 مثل fork()"

الصحيح:

exec() **ما يرجع أبداً** إذا نجح - لأنه استبدل الكود كله!

- يرجع بس إذا **فشل** (-1)

⭐ **نصيحة:** إذا شفت كود بعد exec()، معناها exec() فشل

فخ 3: **Orphan** يسبب مشاكل

الخطأ الشائع:

"Orphan Process مشكلة خطيرة زي Zombie"

الصحيح:

- Orphan **مو مشكلة** - init process يتبناها
- **Zombie** هي المشكلة - تملأ Process Table

فخ 4: **Context Switch** وقت مفيد

الخطأ الشائع:

"Context Switch جزء من تنفيذ العمليات"

الصحيح:

Context Switch هو **Overhead (وقت ضائع)** - ما في شغل حقيقي للمستخدم

⭐ **نصيحة:** كل ما قلّ Context Switch، زاد الأداء

فخ 5: fork() يغير كود العملية

الخطأ الشائع:

"fork() يخلي الابن يشتغل برنامج مختلف"

الصحيح:

- **fork()** بس ينسخ - نفس الكود
- **exec()** هو اللي يغيّر الكود

فخ 6: **Ready Queue** والـ **Waiting Queue** نفس الشي

الخطأ الشائع:

"Ready و Waiting نفس المعنى"

الصحيح:

- **Ready:** تنتظر **CPU** (جاهزة للشغل)
- **Waiting:** تنتظر **I/O** أو حدث

فخ 7: **Long-term Scheduler** موجود بكل الأنظمة

الخطأ الشائع:

"كل نظام تشغيل عنده Long-term Scheduler"

الصحيح:

أغلب الأنظمة الحديثة (Windows, UNIX) **ما عندهم** Long-term - يقبلون كل العمليات

القواعد الذهبية (احفظها!)

1. ⭐ **Job Queue** = الأم الكبيرة - تحتوي **كل العمليات**
2. ⭐ **Context Switch** = Overhead (كل ما قلّ، أحسن!)
3. ⭐ **I/O request** → Running → **Waiting** (مو Ready!)
4. ⭐ **I/O finishes** → Waiting → **Ready** → Running (لازم تمر بـ Ready!)
5. ⭐ **fork()** يرجع: **PID للأب / 0 للابن / -1 للفشل**
6. ⭐ **exec() ما يرجع** إذا نجح (استبدل الكود كله!)
7. ⭐ **Zombie()** = Child terminated + Parent doesn't **wait**
8. ⭐ **Orphan** = Parent terminated + Child still **running**
9. ⭐ **I/O bound processes** → Ready Queue **empty**
10. ⭐ **Fork-and-Exec()** = fork ثم exec() (النمط الكلاسيكي)

الأخطاء الشائعة - تجنبها!

❌ **خطأ 1:** Waiting → Running مباشرة → ✅ **الصحيح:** Waiting → Ready → Running

❌ **خطأ 2:** fork() يغيّر الكود → ✅ **الصحيح:** exec() يغيّر الكود

❌ **خطأ 3:** Orphan مشكلة → ✅ **الصحيح:** Zombie هي المشكلة

❌ **خطأ 4:** Context Switch مفيد → ✅ **الصحيح:** Overhead (ضايع)

❌ **خطأ 5:** exec() يرجع قيمة → ✅ **الصحيح:** ما يرجع إذا نجح

❌ **خطأ 6:** Ready ≠ Waiting **الصحيح:** ✅ → Ready = Waiting

❌ **خطأ 7:** Job Queue = New state فقط → ✅ **الصحيح:** كل الحالات

متى نستخدم إيش؟

```text
fork() → محتاج تنسخ عملية
exec() → محتاج تغيّر البرنامج
CreateProcess() → (Windows) محتاج تنشئ عملية
wait() → محتاج تنتظر الابن
Device Queue → I/O العملية طلبت
Ready Queue → العملية جاهزة للشغل
Long-term Scheduler → محتاج تتحكم بعدد العمليات
Short-term Scheduler → CPU محتاج تختار عملية للـ
Medium-term (Swapping) → الذاكرة امتلأت
```

(IPC) الاتصال بين العمليات

🎯 نظرة سريعة: شنو راح تتعلم؟

IPC هو الموضوع اللي يخلي العمليات "تتحچي" بيناتها وتتعاون. بدون IPC، كل عملية جزيرة معزولة ما منها فايدة.

الأهداف:

- ✅ تفرق بين Independent و Cooperating Processes
- ✅ تفهم نموذجي IPC: Shared Memory و Message Passing
- ✅ تعرف الفرق بين Blocking و Non-blocking
- ✅ تميز بين Direct و Indirect Communication
- ✅ تعرف آليات IPC العملية (Pipes, Sockets, RPC)

مدة المراجعة المقترحة: 50 دقيقة 📝

📖 القسم الأول: المفاهيم الأساسية

🔑 التعريفات المهمة

Independent Processes (عمليات مستقلة):

- **التعريف:** عمليات **ما تشارك بيانات** مع أي عملية ثانية - ما تتأثر ولا تؤثر
- **ببساطة:** مثل شخص يسكن لحاله - ما يحچي وية أحد
- **مثال:** حاسبة (Calculator) تشتغل لحالها بدون تفاعل مع برامج ثانية
- ⚠️ **انتبه:** هاي العمليات **نادرة** - أغلب العمليات متعاونة

Cooperating Processes (عمليات متعاونة):

- **التعريف:** عمليات تقدر **تأثر أو تتأثر** بعمليات ثانية - تشارك بيانات
- **ببساطة:** مثل فريق شغل - يتعاونون ويتبادلون معلومات
- **مثال:** Copy من Word ثم Paste في Telegram
- ⭐ **نقطة ذهبية:** أي عملية تشارك بيانات = Cooperating

أسباب التعاون (لماذا نحتاج Cooperating):

1. **مشاركة المعلومات (Information Sharing):** مثل الـ Clipboard المشترك
2. **تسريع الحسابات (Computation Speedup):** استخدام كل الأنوية (Cores) بالتوازي
3. **النمطية (Modularity):** تقسيم النظام لوحدات صغيرة منفصلة
4. **الراحة (Convenience):** المستخدم يقدر يشتغل على عدة مهام بنفس الوقت

📊 نماذج IPC الأساسية

1. Shared Memory (الذاكرة المشتركة):

- **التعريف:** منطقة ذاكرة مشتركة بين عمليتين أو أكثر - واحدة تكتب والثانية تقرأ
- **ببساطة:** مثل "سبورة" في الصالة - أي واحد يكتب عليها والباقي يقرون
- **الآلية:** نظام التشغيل ينشئ منطقة مشتركة، ثم العمليات تتعامل معها كأنها ذاكرة عادية
- **المميزات:**
  - ✅ **أسرع** طريقة IPC
  - ✅ ما تحتاج تدخل Kernel بعد الإنشاء
  - ✅ وصول مباشر للبيانات (No copying)
- **العيوب:**
  - ❌ المبرمج مسؤول عن **المزامنة (Synchronization)**
  - ❌ خطر **Race Conditions** (كتابة متزامنة)
  - ❌ ما تشتغل على أجهزة مختلفة (Same machine only)

2. Message Passing (تمرير الرسائل):

- **التعريف:** العمليات تتبادل رسائل **بدون** مساحة ذاكرة مشتركة
- **ببساطة:** مثل "صندوق البريد" - واحد يرسل رسالة والثاني يستلمها
- **الآلية:** استخدام **send(message)** و **receive(message)**

- **المميزات:**
  - ✅ **أحسن للأنظمة الموزعة (Distributed Systems)**
  - ✅ يشتغل على أجهزة مختلفة متصلة بشبكة
  - ✅ **النظام يوفر المزامنة** تلقائياً
  - ✅ عزل أفضل بين العمليات (Isolation)
- **العيوب:**
  - ❌ **أبطأ** من Shared Memory
  - ❌ يحتاج تدخل Kernel في كل رسالة (System calls)
  - ❌ يستهلك موارد بسبب نسخ البيانات

📊 جدول المقارنة الشامل (حفظ واجب!)

الميزة	Shared Memory	Message Passing
السرعة	✅ **أسرع**	❌ أبطأ
تدخل Kernel	مرة وحدة (الإنشاء)	كل رسالة (Overhead)
المزامنة	❌ المبرمج مسؤول	✅ النظام يوفرها
الأجهزة	نفس الجهاز فقط	عدة أجهزة (شبكة)
التعقيد	أعقد (Race conditions)	أبسط
نسخ البيانات	❌ لا (وصول مباشر)	✅ نعم (Copying)
الاستخدام الأمثل	Same machine + High performance	Distributed systems

⭐ **للحفظ السريع:**

- Shared Memory = **سريع + نفس الجهاز + تعقيد**
- Message Passing = **بطيء + عدة أجهزة + بساطة**

⚠️ **فخ شائع:** بعض الطلاب يعتقدون Message Passing أسرع - خطأ! Shared Memory أسرع

💡 القسم الثاني: تفاصيل Message Passing

🔑 التسمية (Naming)

1. Direct Communication (اتصال مباشر):

- **التعريف:** كل عملية تذكر اسم (PID) العملية الثانية **صراحة**
- **الصياغة:**
  - `send(P, message)` - أرسل رسالة للعملية P
  - `receive(Q, message)` - استقبل رسالة من العملية Q
- **الخصائص:**
  - Link تلقائي بين كل زوج (Automatic establishment)
  - Link بين **عمليتين فقط** (One-to-one)
  - **بس رابط واحد** بين كل زوج
- ⚠️ **العيب:** لو غيّرت اسم العملية، لازم تعدل كل الكود

2. Indirect Communication (اتصال غير مباشر):

- **التعريف:** الرسائل ترسل لـ **Mailbox (Port)** - أي عملية تشارك بالصندوق تقدر تستلم
- **الصياغة:**
  - `send(A, message)` - أرسل رسالة للصندوق A
  - `receive(A, message)` - استقبل رسالة من الصندوق A
- **الخصائص:**
  - Link ينشأ **فقط** إذا العمليات تتشارك mailbox
  - Link ممكن يكون **بين أكثر من عمليتين** (Many-to-many)
  - **عدة links** بين نفس العمليات (عدة mailboxes)
- ⭐ **الميزة:** مرونة أكبر - العمليات ما تحتاج تعرف بعض مباشرة

🔄 المزامنة (Synchronization)

1. Blocking (Synchronous):

- **Blocking Send:** المرسل **يتوقف (يوكف)** لحد ما المستقبل يستلم الرسالة
- **Blocking Receive:** المستقبل **يتوقف** لحد ما الرسالة توصل
- **ببساطة:** مثل "اتصال التلفون" - لازم تنتظر الرد
- ⭐ **نقطة ذهبية:** Blocking = Synchronous (نفس المعنى)
- **الاستخدام:** لما تحتاج **تأكيد** إن الرسالة وصلت

2. Non-blocking (Asynchronous):

- **Non-blocking Send:** المرسل **يرسل ويكمل** شغله - ما ينتظر
- **Non-blocking Receive:** المستقبل يستلم رسالة (إذا موجودة) أو **null**
- **ببساطة:** مثل "رسالة الواتساب" - ترسل وتروح، ما تنتظر الرد
- ⭐ **نقطة ذهبية:** Non-blocking = Asynchronous
- **الاستخدام:** لما الأداء مهم وما تحتاج تأكيد فوري

⚠️ **فخ شائع:** بعض الطلاب يخلطون بين Blocking و Synchronous - هم **نفس الشي**!

📦 التخزين المؤقت (Buffering)

1. Zero Capacity:

- **التعريف: ما في طابور** - المرسل لازم يوكف لحد ما المستقبل يستلم
- **ببساطة:** "تسليم يد بيد" - ما في مخزن وسيط
- ⭐ **قاعدة:** Zero capacity = **Blocking send إجباري**

2. Bounded Capacity:

- **التعريف:** طابور بحجم **محدد (n messages)**
- **الآلية:**
  - إذا الطابور **مو ممتلئ** → المرسل يرسل ويكمل (Non-blocking)
  - إذا الطابور **ممتلئ** → المرسل يوكف (Blocking)

3. Unbounded Capacity:

- **التعريف:** طابور **غير محدود** - يقدر يستوعب أي عدد رسائل
- ⭐ **قاعدة:** Unbounded = **المرسل ما يوكف أبداً** (Non-blocking always)

🛠️ القسم الثالث: الآليات العملية (Real-world Tools)

🔧 الأنابيب (Pipes)

1. Ordinary Pipes (الأنابيب العادية):

- **التعريف:** قناة اتصال **أحادية الاتجاه (Unidirectional)** بين عمليتين
- **الشرط:** لازم يكون فيه علاقة **Parent-Child**
- **ببساطة:** مثل "صوندة المي" - تمشي باتجاه واحد
- **الاستخدام:**
  - Parent يكتب → Child → Pipe يقرأ
  - أو العكس
- ⚠️ **القيد: ما تشتغل** بين عمليات مو مرتبطة
- **مثال في UNIX:**

```bash
ls | more  # الـ Pipe (|) يمرر output من ls لـ more
```

2. Named Pipes (FIFOs):

- **التعريف:** أنابيب **ثنائية الاتجاه (Bidirectional)** مع اسم في النظام
- **الميزة: ما تحتاج** علاقة Parent-Child - أي عمليتين يقدرون يستخدمونها
- **ببساطة:** مثل "أنبوب بدايتين" - تمشي بالاتجاهين
- **الاستخدام:** لما تحتاج اتصال دائم بين عمليات مختلفة

🌐 المقابس (Sockets)

التعريف:

نقطة نهاية (Endpoint) للاتصال عبر **الشبكة** بين عمليات على أجهزة مختلفة

المكونات:

- **IP Address:** عنوان الجهاز (مثل 192.168.1.10)
- **Port Number:** رقم المنفذ (مثل 80 للـ HTTP)
- **Socket = IP + Port** (مثل: 192.168.1.10:8080)

الأنواع:

- **TCP Sockets:** اتصال موثوق (Reliable) - يضمن وصول البيانات بالترتيب
- **UDP Sockets:** اتصال غير موثوق (Unreliable) - أسرع بس ما يضمن الوصول

المميزات:

- ✅ **ثنائي الاتجاه (Bidirectional)**
- ✅ يشتغل على **أجهزة مختلفة** (Network communication)
- ✅ يدعم **بروتوكولات** (TCP/IP)

⭐ **للامتحان:** Sockets أحسن طريقة للاتصال عبر الشبكة

📞 استدعاء الإجراءات عن بُعد (RPC - Remote Procedure Call)

التعريف:

آلية تخلي البرنامج ينفذ **دالة (Procedure)** موجودة على جهاز ثاني **كأنها محلية**

ببساطة:

مثل لما تتصل بصديقك وتطلب منه يسوي شي بدالك - وكأنك أنت اللي سويته

الآلية:

1. Client يستدعي الدالة (مثل: `add(5, 3)`)
2. RPC يحول الاستدعاء لرسالة ويرسلها عبر الشبكة
3. Server ينفذ الدالة ويرجع النتيجة
4. Client يستلم النتيجة (8)

المميزات:

- ✅ **شفافية (Transparency)**: المبرمج ما يحتاج يعرف إن الدالة على جهاز ثاني
- ✅ مفيد لـ **Distributed Systems**
- ✅ يخفي تفاصيل الشبكة

⚠️ **التحديات:**

- مشاكل الشبكة (Network failures)
- اختلاف أنظمة التشغيل (Heterogeneity)

📊 جدول المقارنة: Pipes vs Sockets vs RPC

الميزة	Ordinary Pipes	Named Pipes	Sockets	RPC
الاتجاه	Unidirectional	Bidirectional	Bidirectional	Bidirectional
الشرط	Parent-Child	لا شرط	لا شرط	لا شرط
الأجهزة	Same machine	Same machine	**Different machines**	**Different machines**
البروتوكول	❌ لا	❌ لا	✅ TCP/IP	✅ نعم
الاستخدام	بسيط ومحلي	محلي ودائم	شبكة	دوال موزعة

⚠️ القسم الرابع: الأخطاء الشائعة والفخوخ

فخ 1: **Message Passing** أسرع من **Shared Memory**

الخطأ الشائع:

"Message Passing أسرع لأنه يستخدم الشبكة"

الصحيح:

Shared Memory أسرع - لأنه وصول مباشر بدون Kernel overhead

فخ 2: **Blocking** و **Synchronous** مختلفين

الخطأ الشائع:

"Blocking شي و Synchronous شي ثاني"

الصحيح:

Blocking = Synchronous (نفس المعنى تماماً)
Non-blocking = Asynchronous

⭐ **نصيحة:** لو شفت "Blocking" بالسؤال، اقرأها "Synchronous"

فخ 3: **Ordinary Pipes** تشتغل بين أي عمليات

الخطأ الشائع:

"Ordinary Pipes تربط أي عمليتين"

الصحيح:

Ordinary Pipes **لازم** Parent-Child relationship
Named Pipes اللي تشتغل بين أي عمليات

فخ 4: **Shared Memory** توفر المزامنة تلقائياً

الخطأ الشائع:

"Shared Memory يحل مشكلة المزامنة تلقائياً"

الصحيح:

المبرمج مسؤول عن المزامنة في Shared Memory
Message Passing هو اللي يوفر المزامنة تلقائياً

فخ 5: Zero Capacity يسمح بـ Non-blocking

الخطأ الشائع:

"كل أنواع Buffering تسمح بـ Non-blocking"

الصحيح:

Blocking = Zero Capacity **إجباري** (ما في مخزن!)

فخ 6: Socket تحتاج نفس الجهاز

الخطأ الشائع:

"Sockets مثل Pipes - بنفس الجهاز بس"

الصحيح:

Sockets **مصممة للشبكة** - أجهزة مختلفة

فخ 7: RPC أبسط من Message Passing

الخطأ الشائع:

"RPC بسيط لأنه مثل استدعاء دالة عادية"

الصحيح:

RPC **أعقد** من الداخل - بس يخفي التعقيد عن المبرمج

التعريفات في 30 ثانية

```text
Independent = ما تشارك بيانات (نادرة)
Cooperating = تشارك بيانات (أغلب العمليات)
Shared Memory = سبورة مشتركة (أسرع)
Message Passing = صندوق بريد (أبطأ + شبكة)
Synchronous = Blocking (ينتظر)
Asynchronous = Non-blocking (ما ينتظر)
Direct = send(P, msg) - اسم صريح
Indirect = send(A, msg) - mailbox
Ordinary Pipes = أحادي + Parent-Child
Named Pipes = ثنائي + أي عمليات
Sockets = IP + Port (شبكة)
RPC = دالة بعيدة (Transparency)
```

القواعد الذهبية (احفظها!)

1. ⭐ **Shared Memory** = **الأسرع** (لكن نفس الجهاز فقط)
2. ⭐ **Message Passing** = **أبطأ** (لكن يشتغل على الشبكة)
3. ⭐ **Synchronous** = **Blocking** (نفس المعنى!)
4. ⭐ **Asynchronous** = **Non-blocking** (نفس المعنى!)
5. ⭐ **Shared Memory** → المبرمج يحل المزامنة
6. ⭐ **Message Passing** → النظام يوفر المزامنة
7. ⭐ **Blocking** = **Zero Capacity إجباري** (ما في مخزن)
8. ⭐ **Ordinary Pipes** = Parent-Child + Unidirectional
9. ⭐ **Named Pipes** = أي عمليات + Bidirectional
10. ⭐ **Socket** = IP + Port (للشبكة)

الأخطاء الشائعة - تجنبها!

❌ **خطأ 1:** Message Passing أسرع → ✅ **الصحيح:** Shared Memory أسرع

❌ **خطأ 2:** Blocking = Synchronous **الصحيح:** ✅ → Blocking ≠ Synchronous

❌ **خطأ 3:** Ordinary Pipes ثنائية → ✅ **الصحيح:** أحادية (Named ثنائية)

❌ **خطأ 4:** Shared Memory يوفر مزامنة → ✅ **الصحيح:** المبرمج مسؤول

❌ **خطأ 5:** Blocking **الصحيح:** ✅ → Zero Capacity = Non-blocking إجباري

❌ **خطأ 6:** Socket = نفس الجهاز → ✅ **الصحيح:** عدة أجهزة (شبكة)

❌ **خطأ 7:** Ordinary Pipes = أي عمليات → ✅ **الصحيح:** Parent-Child فقط

متى نستخدم إيش؟

```text
محتاج سرعة عالية + نفس الجهاز → Shared Memory
محتاج اتصال عبر الشبكة → Message Passing / Sockets
محتاج بساطة + توزيع → Message Passing
محتاج Parent-Child بسيط → Ordinary Pipes
محتاج أي عمليات + ثنائي الاتجاه → Named Pipes
محتاج استدعاء دالة بعيدة → RPC
محتاج IP + Port → Sockets
محتاج مزامنة تلقائية → Message Passing
```

اتصالات العميل-الخادم

🎯 نظرة سريعة: شنو راح تتعلم؟

هذا الموضوع يشرح شلون العمليات "تتحچي" بيناتها حتى لو كانوا على **أجهزة مختلفة** متصلة بشبكة. هاي آخر موضوع بالـ Chapter 3.

الأهداف:

- ✅ تفهم Sockets وتركيبها (IP + Port)
- ✅ تعرف Well-known Ports (< 1024)
- ✅ تفهم RPC وشلون تشتغل
- ✅ تعرف Stubs (Client + Server) ودورهم
- ✅ تفهم Marshaling و XDR
- ✅ تميز بين Big-endian و Little-endian
- ✅ تعرف Binding mechanisms

📖 القسم الأول: المقابس (Sockets)

🔑 التعريف والمفاهيم الأساسية

Socket (المقبس):

- **التعريف:** نقطة نهاية (Endpoint) للاتصال بين جهازين عبر الشبكة
- **ببساطة:** مثل "رقم تلفون داخلي" - IP رقم الشركة، Port رقم التحويلة
- **التركيب:** Socket = **IP Address + Port Number**
- **مثال:** 192.168.1.10:8080

⭐ **نقطة ذهبية:** كل جهاز محتاج socket - العميل عنده socket والخادم عنده socket

⚙️ آلية عمل Socket

الخطوات:

1. **Server:** يفتح socket ويسوي **listen()** على منفذ معين (مثل 80)
2. **Client:** يفتح socket ويتصل بـ IP الخادم + Port
3. **Server:** يسوي **accept()** للاتصال
4. **الاتصال يكتمل:** تبدأ عملية نقل البيانات
5. **الإغلاق:** كلاهما يسوي **close()**

⭐ **للحفظ:** listen → connect → accept → transfer → close

🔢 المنافذ (Ports)

Well-known Ports (المنافذ المشهورة):

- **التعريف:** منافذ تحت رقم 1024 محجوزة للخدمات القياسية
- **أمثلة مهمة للامتحان:**
  - **Port 80:** HTTP (مواقع الويب)
  - **Port 21:** FTP (نقل الملفات)
  - **Port 22:** SSH (الاتصال الآمن)
  - **Port 23:** Telnet
  - **Port 25:** SMTP (البريد الإلكتروني)
  - **Port 443:** HTTPS (مواقع آمنة)

منافذ العميل (Client Ports):

- **القاعدة:** العميل يستخدم منافذ **أكبر من 1024** (عشوائية)
- **السبب:** المنافذ الصغيرة محجوزة للخوادم
- ⚠️ **فخ شائع:** بعض الطلاب يعتقدون العميل يستخدم منافذ < 1024 - خطأ!

📊 خصائص Sockets

المميزات:

- ✅ **Bidirectional** (ثنائي الاتجاه)
- ✅ يشتغل على **أجهزة مختلفة** (Network)
- ✅ يدعم TCP (موثوق) و UDP (سريع)

العيوب:

- ❌ **Low-level** (منخفض المستوى)
- ❌ ينقل **bytes غير منظمة** (Unstructured stream of bytes)
- ❌ **التطبيق مسؤول** عن تفسير البيانات
- ❌ المبرمج يكتب كود شبكة معقد

⚠️ **للامتحان:** Sockets = Low-level + Unstructured data

📞 القسم الثاني: استدعاء الإجراءات عن بُعد (RPC)

🔑 التعريف الأساسي

RPC (Remote Procedure Call):

- **التعريف:** آلية تخلي استدعاء دالة على جهاز بعيد يبان **كأنه استدعاء محلي**
- **ببساطة:** مثل طلب "وجبة رقم 1" من مطعم - ما يهمك شنو يصير بالمطبخ، المهم الوجبة توصل
- **الهدف:** Transparency (شفافية) - المبرمج ما يحتاج يعرف تفاصيل الشبكة

⭐ **قاعدة ذهبية:** High-level = RPC (عكس Sockets)

🛠 مكونات RPC: Stubs (الوكلاء)

Client Stub:

- **الدور:** وكيل العميل - يخفي تفاصيل الشبكة
- **الخطوات:**
  1. يستقبل الـ **parameters** من البرنامج
  2. يسوي **Marshaling** (تحويل البيانات لشكل قابل للنقل)
  3. يرسل الرسالة عبر الشبكة للـ Server
  4. ينتظر الرد
  5. يسوي **Unmarshaling** للنتيجة ويرجعها للبرنامج

Server Stub:

- **الدور:** وكيل الخادم - يستقبل وينفذ
- **الخطوات:**
  1. يستقبل الرسالة من الشبكة
  2. يسوي **Unmarshaling** (فك البيانات)
  3. ينفذ الدالة الحقيقية على الخادم
  4. ياخذ النتيجة ويسوي **Marshaling**
  5. يرجع النتيجة للعميل

⭐ **للحفظ:** Client Stub = المرسل | Server Stub = المنفذ

⚠️ **فخ شائع:** بعض الطلاب يعتقدون Stub يشفر البيانات (Encryption) - خطأ! هو بس يحول الشكل

🌐 XDR (External Data Representation)

التعريف:

تمثيل موحد قياسي للبيانات في RPC - يضمن فهم مشترك بين الأجهزة المختلفة

الهدف:

حل مشكلة اختلاف الأجهزة في تخزين البيانات

الصيغة القياسية:

XDR يستخدم **Big-endian** كصيغة موحدة

⚠️ **للامتحان:** XDR = Standard format للبيانات في RPC

🔄 Endianness (ترتيب البايتات)

المشكلة:

الأجهزة تختلف في ترتيب تخزين البايتات بالذاكرة!

Big-endian:

- **التعريف:** البايت **الأهم** (Most Significant Byte) يخزن في **أصغر عنوان**
- **مثال:** العدد 0x0A0B0C0D يخزن بالترتيب:

```text
Address:
Value:   [0A]  [0B]  [0C]  [0D]
```

- **يستخدمه:** **Network protocols**، Motorola، SPARC
- ⭐ **نقطة ذهبية:** Big-endian = "الكبير أولاً"

Little-endian:

- **التعريف:** البايت **الأقل أهمية** (Least Significant Byte) يخزن في **أصغر عنوان**
- **مثال:** العدد 0x0A0B0C0D يخزن بالترتيب:

```text
Address:
Value:   [0D]  [0C]  [0B]  [0A]
```

- **يستخدمه:** AMD، Intel x86
- ⭐ **نقطة ذهبية:** Little-endian = "الصغير أولاً"

⚠️ **فخ شائع:** بعض الطلاب يخلطون المفهومين - احفظ بالمثال!

⭐ **للامتحان:** XDR يستخدم **Big-endian** كصيغة موحدة

✅ ضمان التنفيذ (Execution Semantics)

المشكلة:

الشبكة غير موثوقة - الرسالة قد تضيع أو تتكرر!

الأنواع:

1. **At least once:**
   - العميل يعيد الإرسال إذا ما وصل رد
   - **المشكلة:** الدالة قد تتنفذ عدة مرات!
   - **مثال خطير:** لو الدالة "اطرح 100 دينار من الحساب"
2. **At most once:**
   - استخدام **Timestamps** لاكتشاف الرسائل المكررة
   - Server يتجاهل الرسائل المكررة
   - **المشكلة:** الدالة قد ما تتنفذ أبداً!
3. **Exactly once (الأفضل):**
   - **الآلية:**
     - العميل يعيد الإرسال دورياً
     - Server يرسل **ACK (تأكيد)** بعد التنفيذ
     - استخدام Timestamps لتجنب التكرار
   - ⭐ **للامتحان:** RPC يحاول يضمن **Exactly once**

🔗 **Binding** (الربط)

المشكلة:

العميل كيف يعرف رقم منفذ (Port) الخادم؟

الحلول:

1. **Fixed Port Binding:**
   - الخادم يستخدم **منفذ ثابت معروف** (Well-known port)
   - **مثال:** Web server دائماً على Port 80
   - **الميزة:** بسيط
   - **العيب:** محدود لخدمات قياسية

2. Dynamic Binding (Matchmaker/Rendezvous):

- **الآلية:**
  1. يوجد **Rendezvous daemon** (خدمة وسيطة) تشتغل على منفذ معروف
  2. Server يسجل نفسه عند الـ Rendezvous (مع رقم منفذه)
  3. Client يسأل Rendezvous: "وين أكدر ألكي الخدمة X؟"
  4. Rendezvous يرد: "الخدمة X على Port 5432"
  5. Client يتصل بالـ Server مباشرة
- **الميزة:** مرونة أكبر - المنافذ تتغير ديناميكياً
- ⭐ **للامتحان:** Matchmaker = دليل الخدمات

📱 Android RPC

Binder Framework:

- آلية RPC الخاصة بنظام Android
- **الاستخدام:** IPC على **نفس الجهاز** (مو شبكة!)
- **الهدف:** اتصال بين Apps و Services

AIDL (Android Interface Definition Language):

- لغة لتعريف الواجهات (Interfaces)
- تحدد الدوال اللي Client يقدر يطلبها من Service
- **ببساطة:** مثل "القائمة" - تحدد شنو الخدمات المتاحة

⚠️ **انتبه:** Android RPC ≠ Network RPC (على نفس الجهاز بس)

📊 القسم الثالث: المقارنات الشاملة

جدول المقارنة: Sockets vs RPC

الميزة	Sockets	RPC
المستوى	Low-level	**High-level**
التعقيد	معقد (المبرمج يكتب كود شبكة)	بسيط (شفافية)
البيانات	Unstructured bytes	Structured parameters
الاستخدام	نقل بيانات خام	استدعاء دوال بعيدة
المرونة	✅ عالية	محدودة بالدوال المحددة
الأداء	أسرع (أقل overhead)	أبطأ (Stubs + Marshaling)
الاستخدام الأمثل	Streaming، Gaming	Distributed systems

جدول Big-endian vs Little-endian

الميزة	Big-endian	Little-endian
الترتيب	الكبير أولاً (MSB first)	الصغير أولاً (LSB first)
مثال 0x0A0B0C0D	0A → 0B → 0C → 0D	0D → 0C → 0B → 0A
الأجهزة	SPARC، Motorola	Intel x86، AMD
الشبكات	✅ يستخدمه (Network byte order)	❌ لا
XDR	✅ الصيغة القياسية	يحتاج تحويل

⚠️ القسم الرابع: الأخطاء الشائعة

فخ 1: Client يستخدم Well-known ports

الخطأ: "العميل يستخدم منافذ < 1024"

الصحيح: العميل يستخدم منافذ > **1024** (عشوائية)

فخ 2: Marshaling = Encryption

الخطأ: "Marshaling يشفر البيانات"

الصحيح: Marshaling **يحول شكل** البيانات، مو تشفير

فخ 3: RPC أسرع من Sockets

الخطأ: "RPC أسرع لأنه High-level"

الصحيح: Sockets **أسرع** - RPC عنده overhead (Stubs + Marshaling)

فخ 4: Little-endian هو الصيغة القياسية

الخطأ: "Little-endian يستخدمه XDR"

الصحيح: Big-endian هو الصيغة القياسية للشبكات

فخ 5: Android RPC للشبكات

الخطأ: "Android RPC يشتغل بين أجهزة مختلفة"

الصحيح: Android RPC على **نفس الجهاز** بس

التعريفات في 30 ثانية

```text
Socket = IP + Port (Endpoint)
Well-known ports = < 1024 (للخدمات)
Client ports = > 1024 (عشوائية)
Port 80 = HTTP | Port 21 = FTP | Port 443 = HTTPS
Sockets = Low-level + Unstructured bytes
RPC = High-level + Transparency
Client Stub = Marshaling (تجميع)
Server Stub = Unmarshaling + Execute (فك وتنفيذ)
Marshaling = تحويل الشكل (مو تشفير!)
XDR = صيغة موحدة (Big-endian)
Big-endian = الكبير أولاً (MSB first)
Little-endian = الصغير أولاً (LSB first)
Exactly once = Timestamps + ACK
Matchmaker = دليل الخدمات (Binding)
Android RPC = Binder + AIDL (نفس الجهاز)
```

القواعد الذهبية

1. ⭐ **Socket** = IP + Port (نقطة نهاية)
2. ⭐ **Well-known ports** < 1024 (للخوادم)
3. ⭐ **Client ports** > 1024 (عشوائية)
4. ⭐ **Sockets** = Low-level + Fast + Unstructured
5. ⭐ **RPC** = High-level + Simple + Transparency
6. ⭐ **Marshaling** = تحويل الشكل (مو تشفير!)
7. ⭐ **XDR** يستخدم **Big-endian**
8. ⭐ **Big-endian** = الكبير أولاً | Little = الصغير أولاً
9. ⭐ **Exactly once** = أفضل execution semantic
10. ⭐ **Matchmaker** = دليل يربط Client بـ Server

المنافذ المشهورة (احفظها!)

```text
Port 21 = FTP (نقل ملفات)
Port 22 = SSH (اتصال آمن)
Port 23 = Telnet
Port 25 = SMTP (بريد)
Port 80 = HTTP (ويب)
Port 443 = HTTPS (ويب آمن)
```